Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Department of CSE

B.Tech. CSE Trimester -5

Roll Number:2520030011
Name:P.Tejaswini

Generative AI and AI Platforms

25CS2207E

WEEK-2: PROVIDER-AGNOSTIC LLM APPLICATION

AIM

To implement a provider-agnostic LLM application using Gemini and Ollama with a common interface in Python.

REQUIREMENTS

- Python
- PyCharm
- Ollama with Llama 3.2
- Gemini API
- Python packages: `google-genai`, `requests`, `python-dotenv`, `pytest`

PROCEDURE

1. Created a Python project named `Week-2` in PyCharm and configured a virtual environment.
2. Installed the required Python packages using pip.
3. Created `llm_client.py`, `main.py`, `test_client.py`, `.env`, and `requirements.txt`.
4. Implemented an abstract `LLMProvider` class with a `generate()` method.
5. Implemented `GeminiProvider` to generate responses using the Gemini API.
6. Implemented `OllamaProvider` to generate responses using the local Llama 3.2 model.
7. Implemented `LLMClient` to communicate with different providers through a common interface.
8. Executed `main.py`, selected Ollama, and entered the prompt "What is AI?"
9. Successfully received the response from the Llama 3.2 model.

Source Code:

main.py:

```
import os
from dotenv import load_dotenv

from llm_client import (
    GeminiProvider,
    OllamaProvider,
    LLMClient
)

load_dotenv(override=True)

provider_name = input(
    "Choose provider (gemini/ollama): "
).strip().lower()


if provider_name == "gemini":

    api_key = os.getenv("GEMINI_API_KEY")

    if not api_key:
        raise ValueError("Gemini API key not found in .env")
```

```
    provider = GeminiProvider(api_key)

elif provider_name == "ollama":

    provider = OllamaProvider("llama3.2")

else:

    raise ValueError("Invalid provider selected")



client = LLMClient(provider)

prompt = input("Enter your question: ")

answer = client.generate(prompt)

print("\nLLM Response:")
print(answer)
```

test_client.py:

```
import unittest

from llm_client import LLMClient



# Fake provider for testing
class FakeProvider:

   def generate(self, prompt):
       return "AI stands for Artificial Intelligence."



class TestLLMClient(unittest.TestCase):

   def test_generate(self):

       provider = FakeProvider()
       client = LLMClient(provider)

       result = client.generate("Explain AI")

       self.assertEqual(
           result,
           "AI stands for Artificial Intelligence."
```

```
        )



if __name__ == "__main__":
    unittest.main()
```

llm_client.py:

```
from abc import ABC, abstractmethod
from google import genai
import requests



# Common interface
class LLMProvider(ABC):

    @abstractmethod
    def generate(self, prompt):
        pass



# Gemini cloud provider
class GeminiProvider(LLMProvider):

    def __init__(self, api_key):
        self.client = genai.Client(api_key=api_key)
```

```
    def generate(self, prompt):
        response = self.client.models.generate_content(
            model="gemini-3.8-flash",
            contents=prompt
        )
        return response.text


# Ollama local provider
class OllamaProvider(LLMProvider):

    def __init__(self, model="llama3.2"):
        self.model = model
        self.url = "http://localhost:11434/api/generate"

    def generate(self, prompt):
        response = requests.post(
            self.url,
            json={
                "model": self.model,
                "prompt": prompt,
                "stream": False
            },
            timeout=120
        )
```

```
        response.raise_for_status()
        return response.json()["response"]



# Common client
class LLMClient:

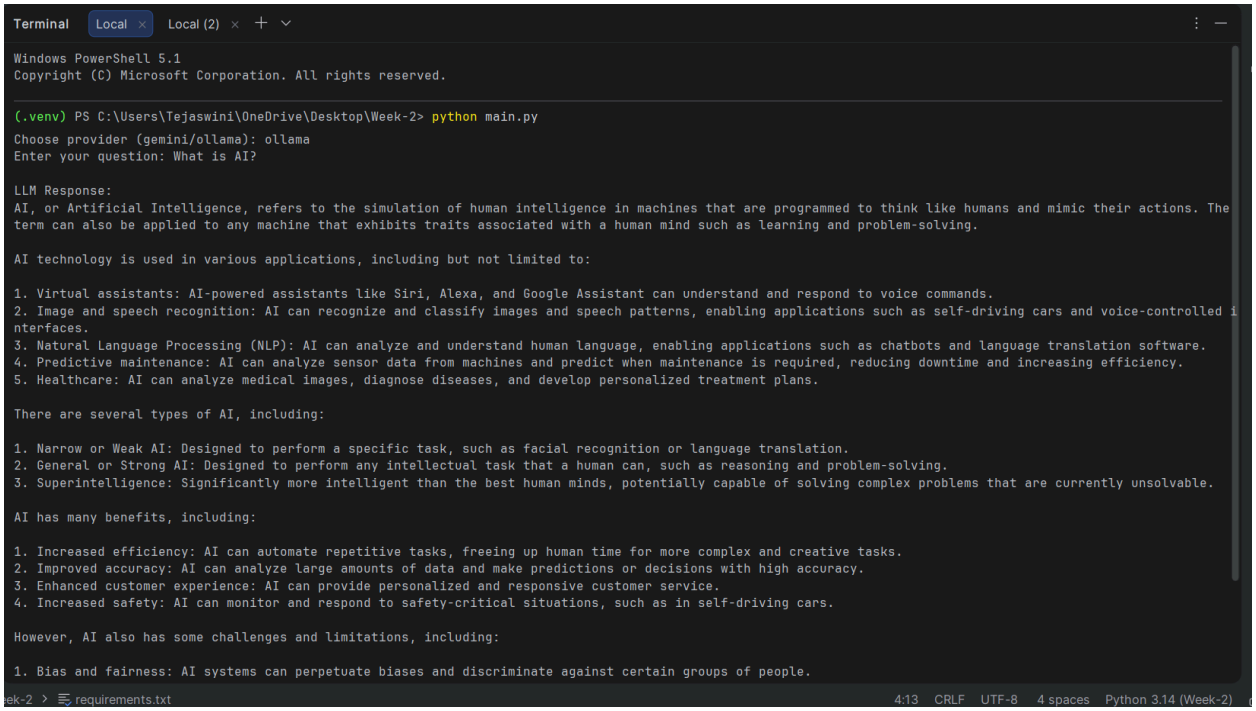

    def __init__(self, provider):
        self.provider = provider


    def generate(self, prompt):
        return self.provider.generate(prompt)
```

requirements.txt:

```
python-dotenv
pytest
google-genai
requests
```

OUTPUT:



RESULT

Successfully implemented a provider-agnostic LLM application using Gemini and Ollama. The Ollama provider generated responses successfully through the common LLMClient interface.